```
# Task1 - Website Visit Analyzer
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pages = []
visits = []

n = int(input("Enter number of pages: "))
for i in range(n):
    p = input("Enter page name: ")
    v = int(input("Enter visit count: "))
    pages.append(p)
    visits.append(v)

df1 = pd.DataFrame({"Page": pages, "Visits": visits})
total_visits = np.sum(visits)

print("\nTotal Visits:", total_visits)

plt.figure()
plt.pie(visits, labels=pages, autopct='%1.1f%%')
plt.title("Website Visits Distribution")
plt.show()

# Task2 - Delivery Time Tracker
order = []
time = []

n = int(input("\nEnter number of orders: "))
for i in range(n):
    o = input("Enter Order ID: ")
    t = int(input("Enter delivery time: "))
    order.append(o)
    time.append(t)
```

```
df2 = pd.DataFrame({"Order": order, "Time": time})
avg_time = np.mean(time)

print("Average Delivery Time:", avg_time)

plt.figure()
plt.plot(order, time, marker='o')
plt.title("Delivery Time")
plt.show()

# Task3 - Employee Salary Analyzer
names = []
salary = []

n = int(input("\nEnter number of employees: "))
for i in range(n):
    n1 = input("Enter employee name: ")
    s = int(input("Enter salary: "))
    names.append(n1)
    salary.append(s)

df3 = pd.DataFrame({"Name": names, "Salary": salary})

total_salary = np.sum(salary)
avg_salary = np.mean(salary)

print("Total Salary:", total_salary)
print("Average Salary:", avg_salary)

plt.figure()
plt.bar(names, salary)
plt.title("Employee Salaries")
plt.show()
```

```
# Task4 - Product Price Comparison
products = []
prices = []

n = int(input("\nEnter number of products: "))
for i in range(n):
    p = input("Enter product name: ")
    pr = int(input("Enter price: "))
    products.append(p)
    prices.append(pr)

df4 = pd.DataFrame({"Product": products, "Price": prices})

print("Highest Price:", np.max(prices))
print("Lowest Price:", np.min(prices))
print("Average Price:", np.mean(prices))

plt.figure()
plt.bar(products, prices)
plt.title("Product Prices")
plt.show()

# Task5 - Rainfall Data Analyzer
days = []
rain = []

n = int(input("\nEnter number of days: "))
for i in range(n):
    d = input("Enter day: ")
    r = int(input("Enter rainfall: "))
    days.append(d)
    rain.append(r)

df5 = pd.DataFrame({"Day": days, "Rainfall": rain})
```

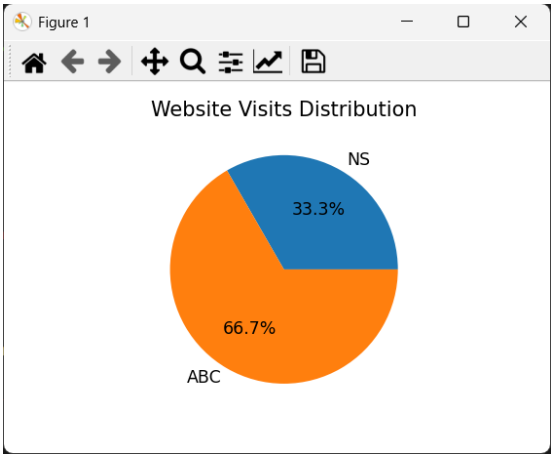
```
print("Total Rainfall:", np.sum(rain))
print("Average Rainfall:", np.mean(rain))

plt.figure()
plt.plot(days, rain)
plt.title("Rainfall Data")
plt.show()

# Task6 - Electricity Usage Tracker
days = []
units = []

n = int(input("\nEnter number of days: "))
for i in range(n):
    d = input("Enter day: ")
    u = int(input("Enter units used: "))
    days.append(d)
    units.append(u)

df6 = pd.DataFrame({"Day": days, "Units": units})

print("Total Usage:", np.sum(units))
print("Average Usage:", np.mean(units))

plt.figure()
plt.plot(days, units)
plt.title("Electricity Usage")
plt.show()

# Task7 - Mobile Usage Analyzer
apps = []
usage = []

n = int(input("\nEnter number of apps: "))
for i in range(n):
```

```
    a = input("Enter app name: ")
    u = int(input("Enter usage time: "))
    apps.append(a)
    usage.append(u)

df7 = pd.DataFrame({"App": apps, "Usage": usage})

print("Total Screen Time:", np.sum(usage))
print("Most Used App:", apps[np.argmax(usage)])

plt.figure()
plt.pie(usage, labels=apps, autopct='%1.1f%%')
plt.title("Mobile Usage")
plt.show()

# Task8 - Sales Performance Tracker
products = []
sales = []

n = int(input("\nEnter number of products: "))
for i in range(n):
    p = input("Enter product name: ")
    s = int(input("Enter sales amount: "))
    products.append(p)
    sales.append(s)

df8 = pd.DataFrame({"Product": products, "Sales": sales})

print("Total Sales:", np.sum(sales))
print("Best Product:", products[np.argmax(sales)])

plt.figure()
plt.bar(products, sales)
plt.title("Sales Performance")
plt.show()
```

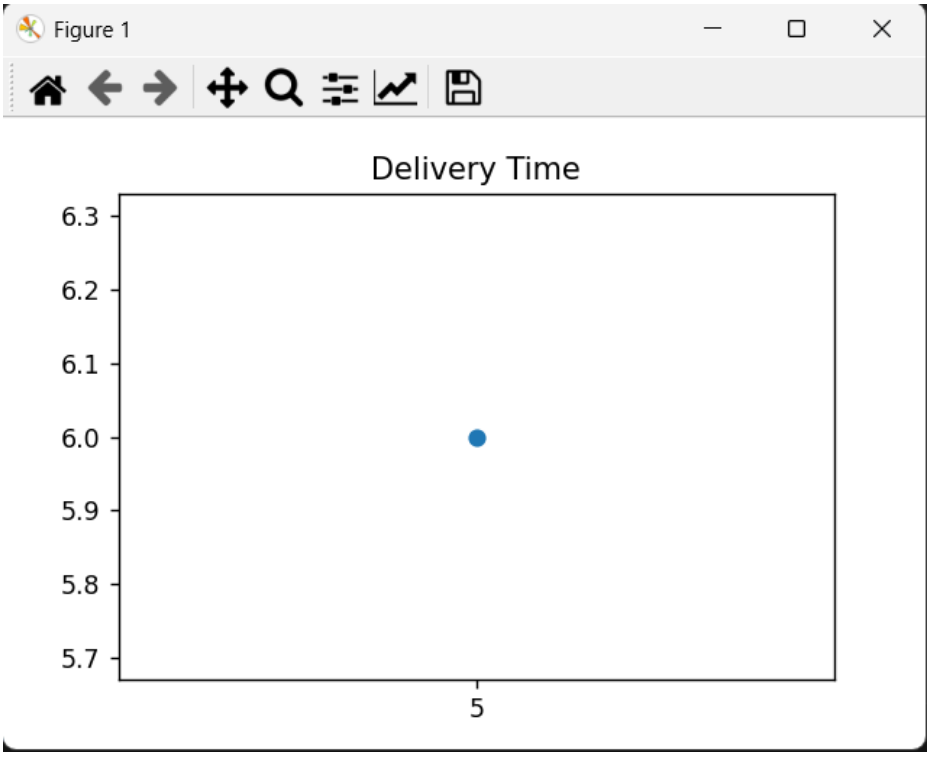
```
# Task9 - Disease Prediction Analyzer
TP = 70
TN = 50
FP = 20
FN = 10

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = 2 * (precision * recall) / (precision + recall)

print("\nAccuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

matrix = pd.DataFrame([[TP, FP],[FN, TN]],
                      columns=["Predicted Yes", "Predicted No"],
                      index=["Actual Yes", "Actual No"])

print("\nConfusion Matrix:\n", matrix)
```

Outputs:

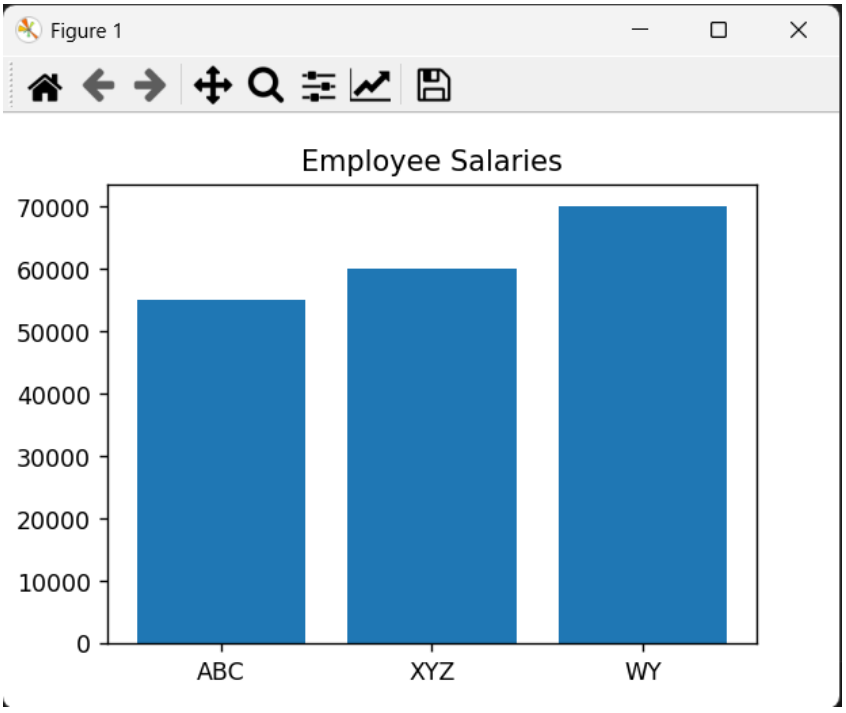
```
Enter number of pages: 2
Enter page name: NS
Enter visit count: 2
Enter page name: ABC
Enter visit count: 4

Total Visits: 6
```



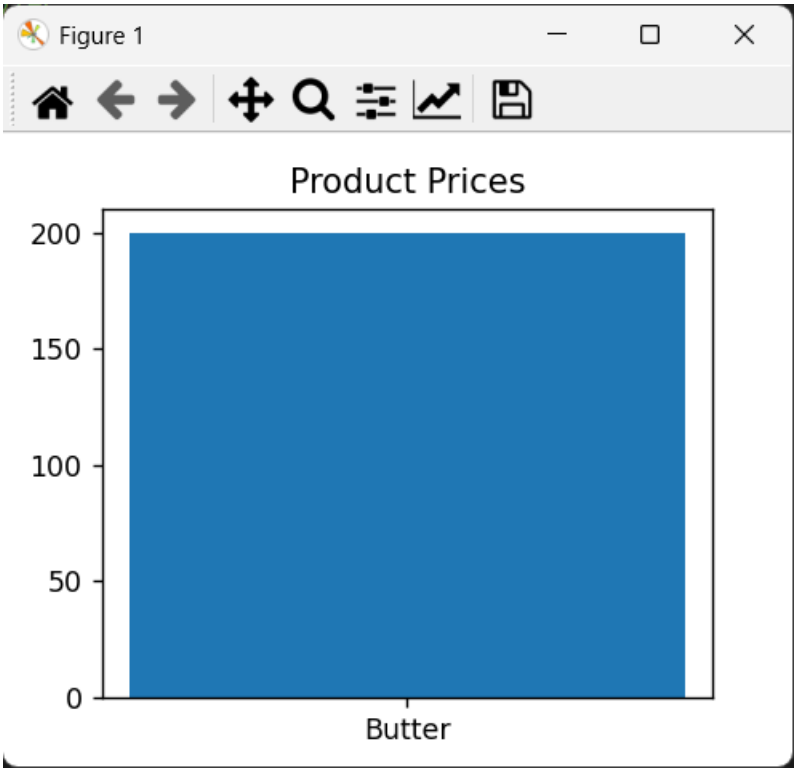
```
Enter number of orders: 1
Enter Order ID: 5
Enter delivery time: 6
Average Delivery Time: 6.0
```



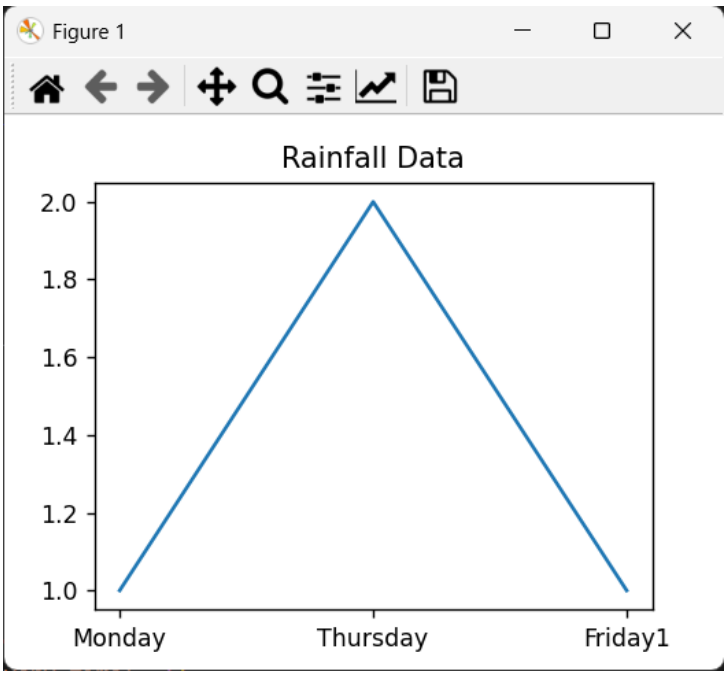

```
Enter number of employees: 3
Enter employee name: ABC
Enter salary: 55000
Enter employee name: XYZ
Enter salary: 60000
Enter employee name: WY
Enter salary: 70000
Total Salary: 185000
Average Salary: 61666.666666666664
```



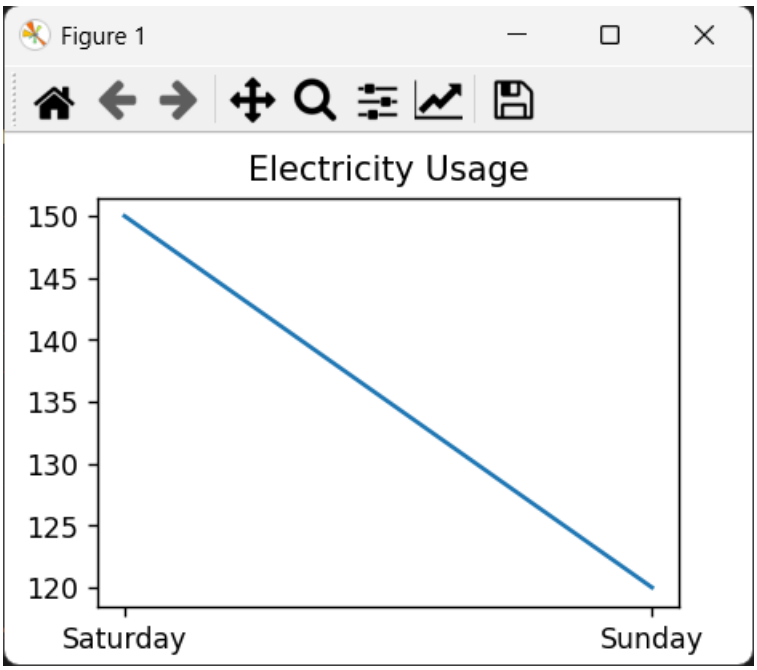
```
Enter number of products: 1
Enter product name: Butter
Enter price: 200
Highest Price: 200
Lowest Price: 200
Average Price: 200.0
```



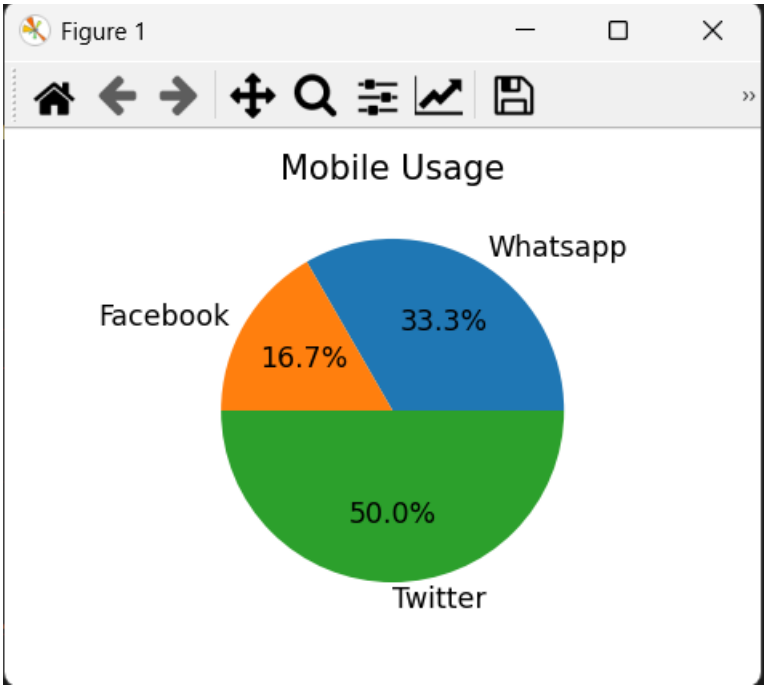

```
Enter number of days: 3
Enter day: Monday
Enter rainfall: 1
Enter day: Thursday
Enter rainfall: 2
Enter day: Friday1
Enter rainfall: 1
Total Rainfall: 4
Average Rainfall: 1.3333333333333333
```



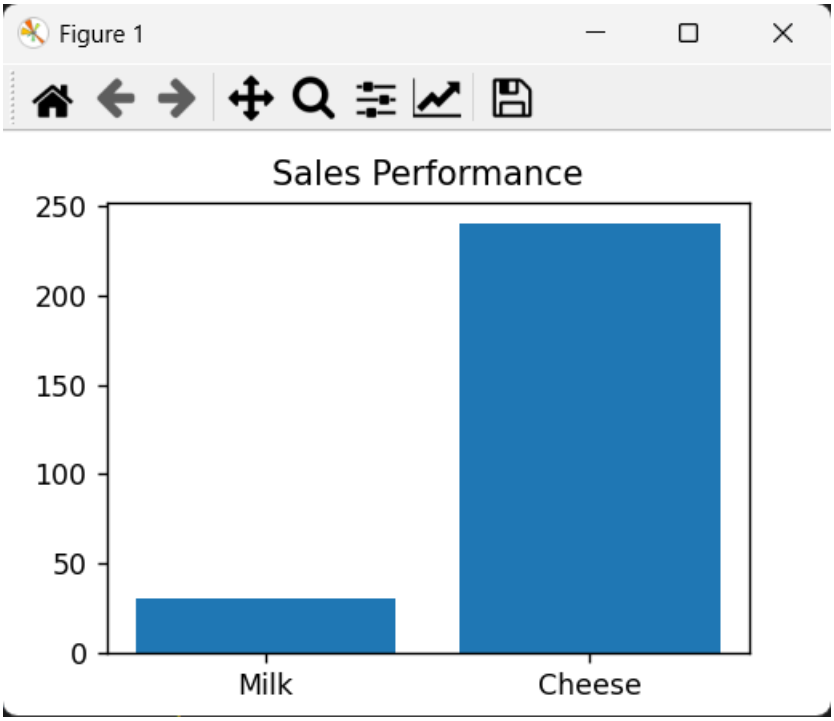
```
Enter number of days: 2
Enter day: Saturday
Enter units used: 150
Enter day: Sunday
Enter units used: 120
Total Usage: 270
Average Usage: 135.0
```




```
Enter number of apps: 3
Enter app name: Whatsapp
Enter usage time: 2
Enter app name: Facebook
Enter usage time: 1
Enter app name: Twitter
Enter usage time: 3
Total Screen Time: 6
Most Used App: Twitter
```



```
Enter number of products: 2
Enter product name: Milk
Enter sales amount: 30
Enter product name: Cheese
Enter sales amount: 240
Total Sales: 270
Best Product: Cheese
```




```
Accuracy: 0.8
Precision: 0.7777777777777778
Recall: 0.875
F1 Score: 0.823529411764706

Confusion Matrix:
             Predicted Yes  Predicted No
Actual Yes              70            20
Actual No               10            50
```